



“Improving Web Automation with DOM Delta Processing and Privacy-Aware LLM Agents”

Major Project – 1

Computer Science & Engineering (Data Science)

—**Darji Dhruv S. (24MCD003)**

—**Mentor: Dr. Parita Oza**

Contents

01 Introduction

02 Problem Statement

03 Literature Review

04 Research Gaps

05 Objective

06 SeeAct: Base Paper

07 Dataset

08 Novelty & Proposed Work

09 Implementation

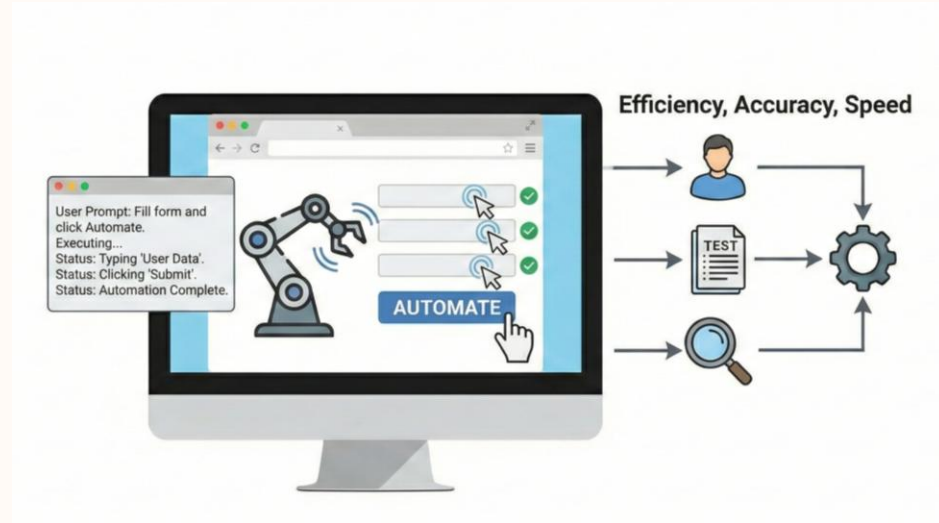
10 Future Work

11 Conclusion

12 References

Introduction

- GUI automation is a way to automate human interaction such as click, type, scroll, input etc. with user given instruction.
- GUI automation accelerates tasks in **testing**, **verification**, **bug replay**, and **text-input generation**, reducing human effort and improving reliability.
- Natural language interfaces allow users to describe tasks ("Order grocery", "Change Settings"), enabling automatic execution by intelligent agents.
- Automation is essential because manual GUI operations are slow, repetitive, error-prone, and difficult to scale.



Problem Statement

- Although recent research has been done on the development of the LLM based web agent, web agents have low robustness, poor privacy, and improper scaling.
- The majority of these approaches are based on complete system summarization of all pages in HTML and code-generation, which is computationally expensive and does not provide cost effective solution.
- As result these agents often fail to generalize across diverse domain or diverse task.
- Which showcase the clear need for more efficient, reliable & scalable, robust and privacy aware web automation methods which are easy to access.

Literature Review

Year	Title	Key Contribution	Limitations
<u>May, 2025</u> [1]	Large Language Model- Brained GUI Agents: A Survey	Systematic , detailed survey of LLM-brained GUI agents: historical evolution, core components, datasets, evaluation metrics, applications. acts as a “cookbook” for building GUI agents.	Identifies open research gaps : robust grounding, benchmarks, long-horizon planning.
<u>Jun, 2025</u> [2]	A Survey on (M)LLM- Based GUI Agents	Breaks down modern GUI agents into four components (perception, exploration, planning, interaction) and analyzes technical/evaluation limitations.	Notes methodological limitations in current benchmarks and evaluation practices also highlight for standardization and improved element localization.
<u>Aug, 2025</u> [3]	A Survey of WebAgents: Towards Next-Generation AI Agents for Web Automation with Large Foundation Models	Reviews WebAgent architectures, training strategies, and trustworthiness. frames web automation as a productivity/UX problem enabled by LFM.	Highlights trustworthiness and robustness issues, and the need for better training/regulatory practices for WebAgents.

Literature Review

Year	Title	Key Contribution	Limitations
Feb, 2025 [4]	GUI Agents with Foundation Models: A Comprehensive Survey	Consolidates recent work using (M)LLMs for GUI tasks. offers taxonomy, datasets review, and unified framework.	Discusses practical gaps like unavailable checkpoints, inconsistent experimental settings, and reproducibility challenges.
Mar, 2024 [5]	GPT-4V(ision) is a Generalist Web Agent, if Grounded	Shows GPT-4V (multimodal LMM) can act as a generalist web agent when grounded with both screenshot + HTML; introduces SeeAct, evaluates on MIND2WEB (51.1% success with manual grounding).	grounding strategies are imperfect and there's a significant gap to oracle grounding. SeeAct's best gains rely on expensive LMMs and manual grounding infrastructure.
Nov, 2024 [6]	AdaptAgent: Adapting Multimodal Web Agents with Few-Shot Learning from Human Demonstrations	Proposes few-shot adaptation using human demonstrations to improve web agents' generalization on unseen sites. shows 3.36%–7.21% absolute gains (relative 21%–66%) on benchmarks.	Improvement depends on demonstration selection and model type. few-shot gains vary and do not fully close the generalization gap for proprietary or very different sites.

Literature Review

Year	Title	Key Contribution	Limitations
<u>May, 2025</u> [7]	LiteWebAgent: The Open-Source Suite for VLM-Based Web-Agent Applications	Provides an open, modular framework (planning, memory, tree search) plus production demos (serverless backend or Chrome extension) to help researchers build and deploy VLM web agents.	actual performance depends on chosen models and grounding modules.
<u>Aug, 2024</u> [8]	OpenWebAgent: An Open Toolkit to Enable Web Agents on Large Language Models	Delivers a modular toolkit with an HTML parser, action generation module, and UI to accelerate building of LLM/LMM web agents; open-source code & plugin.	Toolkit-oriented – does not itself claim new agent performance. Practical utility depends on integration choices and model quality.
<u>Jul, 2025</u> [9]	Browsing Like Human: A Multimodal Web Agent with Experiential Fast-and-Slow Thinking (WebExpert)	Proposes a human-like fast/slow planning paradigm and experiential learning (reflecting on failures) to improve decomposition of complex instructions – shows strong results on MIND2WEB.	Added complexity (planning + reflection) may increase compute and implementation complexity.

Research Gaps

Unreliable Element Grounding

- Current agents struggle with fine-grained visual-to-HTML grounding, causing major failures on real websites [4] [5] [9].

High Cost & Diversity Bottlenecks

- High cost & resource demands, limit wider adoption of (M)LLM-based GUI agents [2].

Privacy and Human-in-Loop Limitations

- Surveys highlight agents often require access to sensitive user data to effectively interact with the GUI environment [1] [3].

Poor Generalization to Unseen Websites

- performance drops sharply for long tasks or unfamiliar layouts [6].
- Overall low success rate over unseen website (<25%) [6].

Large and complex DOMs, missing js, css

- Real-world website contain thousands of DOM element.
- Model does not look JS or CSS while inference, thus result may poor. [4]

Safety, Security & Trustworthiness

- GUI agents remain vulnerable to adversarial UIs or unexpected DOM changes [8].
- lack of standardized safety protocols, privacy-aware design [2].

Objectives

- Build a web automation agent using multimodal which can perform better on some parameter to improve the overall automation process.
- Use HTML summarizer which can parse only query related values further we can use that as input to large models.
- Evaluate zero-shot and few-shot prompting for improving action prediction and grounding.
- Design a fallback for improvement over failure step and agent can go on back step for overall task success.
- Design a HITL for human intervention while Web Automation work on privacy concern data (e.g. while login user must input their credential by their own).

SeeAct: Base Paper

Paper-Title: GPT-4V(ision) is a Generalist Web Agent, if Grounded

- SeeAct use large multimodal model (e.g. GPT-4V(ision)) to see the webpage using screenshot + DOM and reason like a human.
- It demonstrates that visual reasoning + DOM grounding leads to significantly higher real-world task success (51.1% on live websites).
- The paper provides a clear, simple and effective modular pipeline (perception → reasoning → grounding → execution) that serves as a strong baseline for designing our own system.
- SeeAct explicitly highlights element grounding as the key challenge, which directly aligns with the research gap for our project targets.

SeeAct: MultiModal-Mind2Web

What is Multimodal-Mind2Web:

- It is the multimodal version of Mind2Web, Where each webpage state has both screenshot image + HTML DOM [10].
- Covers 137 websites, spanning 31 top-level domains, with a broad variety of user tasks + interaction patterns [10].
- Consists of over 2,000+ distinct tasks (in the original Mind2Web) with crowdsourced action sequences [10].

Split / Set	No. of actions / steps	What it means
Train	7775 actions from 1009 tasks	training pool for learning / experimentation
Test_task	1339 actions from 177 tasks	tests generalization across tasks within known websites
Test_website	1019 actions from 142 tasks	tests cross-website generalization
Test_domain	4060 actions from 694 tasks	tests cross-domain robustness

SeeAct: MultiModal-Mind2Web

What each data-row contains:

- **screenshot**: png snapshot of rendered page before action.
- **raw_html / cleaned_html**: HTML source of the page at that moment.
- **operation**: what action was taken (Click / Type / Select) + optional value (text, dropdown option, etc.)
- **annotation_id**: groups actions belonging to the same high-level “task” (so that multi-step trajectories are reconstruct able).

Why it's suited for Vision-Based Web Agents like SeeAct:

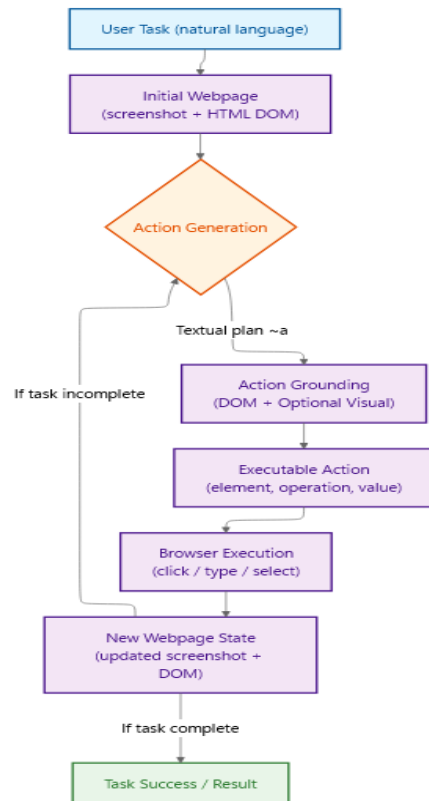
- Dataset is available at 13.8GB of total size, which enable easy development.
- each state has both visual (screenshot) and structural (DOM / HTML) information enabling agents to reason both visually (what user sees) and precisely (which element to click).
- Most benchmarks use this dataset, which leads better comparability.

SeeAct: Architecture & Implementation

Action Generation (LMM): Given the current screenshot + task + history to model which return a textual action description for action.

Action Grounding: Map textual description to actual DOM element + operation (click / type / select) by heuristic ranking of DOM elements.

Browser Execution: Use automation (via Playwright / browser engine) to execute the action. Webpage updates and we get new screenshot + DOM.



SeeAct: Methodology

Paper-Title: GPT-4V(ision) is a Generalist Web Agent, if Grounded

Action Generation: SeeAct uses a large multimodal model (LMM), primarily GPT-4V(ision).

- Screenshot image (visual input), The task description (natural language) and History of previous actions (what has already been done) given to the model.

Action Grounding: SeeAct uses DeBERTa model for grounding with cross-encoder variant, meaning: given a pair of (query, candidate element representation), the model outputs a score representing how likely that candidate is relevant to the query (i.e. the next action).

- Practically: “query” could be some features derived from the textual intention (e.g. “I want to click the button that says ‘Search’”), candidate element = some DOM element represented by its HTML metadata (tag, text content, attributes). The cross-encoder concatenates both and outputs a relevance score.
- They use this to rank all DOM elements and choose top-50 for grounding module. This greatly reduces complexity compared to considering full DOM.

SeeAct: Grounding Methods

Method	What model asks	How grounding works
Element Attributes	Ask LMM to output type + visible text + detailed description of target element (e.g. "button labeled 'Search' near top-right")	Heuristic match over DOM: search for element(s) with that type + text; if unique → select; if ambiguous → further prompt.
Textual Choices	Provide candidate list (top-50) as multi-choice options (HTML snippet + text) to LMM; ask: "Which element corresponds to your intended target?"	LLM picks one choice (or "None"). This directly yields element. Best among automatic methods.
Image Annotation	Overlay bounding boxes (or numbered labels) on the screenshot image, corresponding to candidate elements. feed annotated image + textual plan to LMM; ask it to output label/box number	The box number indicates which element to act upon. This tries to use visual alignment.

SeeAct: Evaluation Strategy & Metrics

Two evaluation settings:

- *Offline evaluation*: on webpages and state-action directly from the dataset (Multimodal-Mind2Web).
- *Online evaluation*: running the agent on live websites through a browser automation tool (Playwright) to check real-world behavior.

Step-level Metrics :

- Element Accuracy: Did the agent pick the correct HTML element vs ground-truth?
- Operation F1: Token-level F1 score of the predicted operation compared to ground truth.
- Step Success Rate: A step is successful only if both element selection and operation are correct. Macro-averaged across tasks.

Task-level Metric:

- Task Success Rate: A full task is counted successful only if final outcome achieved. (Strict: no step-error tolerated).

SeeAct: Evaluation Strategy & Metrics

- Online (Live) Task Success: measure the proportion of tasks the agent fully completes under real conditions. Example: with oracle grounding, SeeAct achieves ~51.1% success on live websites.
- Why multiple metrics: Step-level metrics (Ele. Acc, Op. F1, Step SR) allow diagnosing where failures happen (wrong click, wrong type, wrong element). The Task-level success rate tests real end-to-end performance that whether the agent actually accomplishes full user-specified tasks.

Metric	Purpose / What it measures
Element Accuracy	Correctness of element selection (which DOM node)
Operation F1	Correctness of action type & input value (click, type, select + typed value)
Step Success Rate	Overall correctness of each action step (element + operation)
Task Success Rate (SR)	Whether task defined goal achieve
Online Task Success (Live)	Real-world success executing on live websites

SeeAct: Results, Challenges & Limitations

Strengths

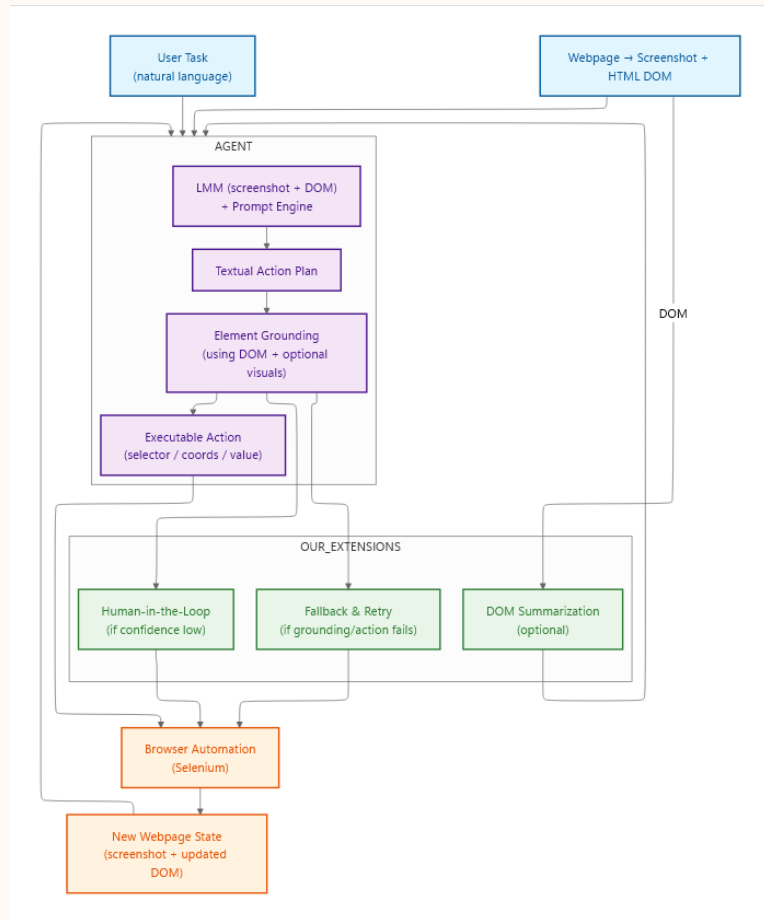
- With oracle grounding (i.e. perfect element mapping), GPT-4V + SeeAct achieves ~51.1% success rate on live websites (end-to-end tasks).
- Demonstrates that vision + language multimodal models are viable as generalist web agents — outperforming text-only models.
- Enables agentic planning, page-reasoning, handling multi-step workflows across diverse, real-world websites.

Limitations

- Grounding (element localization) remains major bottleneck, SeeAct uses heuristic grounding which creates bottlenecks.
- Visual grounding methods (e.g. bounding boxes) struggle on complex, dense web layouts — hallucination, mis-alignment, ambiguous elements.
- Many real-world UI challenges remain: dynamic content, pop-ups, JS-heavy interactions, complex widgets (scroll, hover, drag, modals). SeeAct's action set is limited (click/type/select).

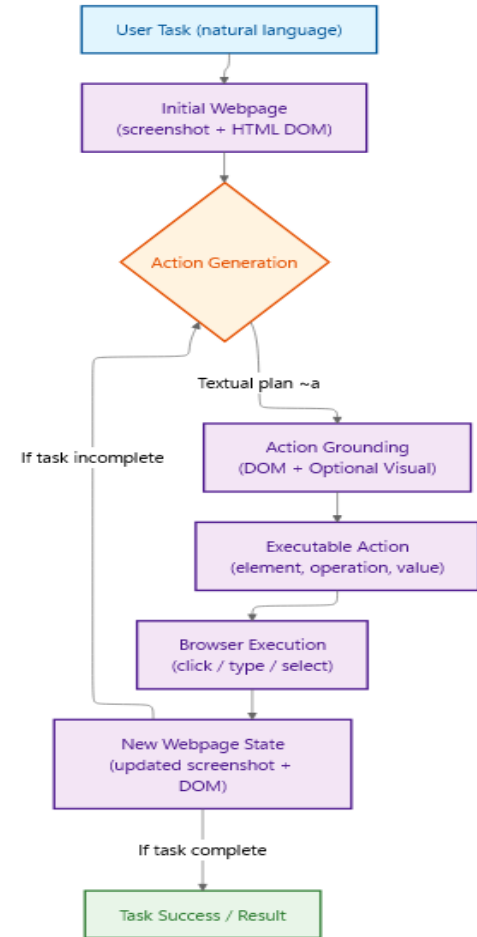
Novelty and Proposed Work

- **Multimodal Input:** Combine Screenshot + HTML DOM during model inference to give both visual and structural context.
- **DOM Summarization:** Apply HTML summarization techniques to compress DOM structure so the model receives meaningful, token-efficient input.
- **Zero-Shot & Few-Shot Prompting:** Implement both techniques to test how example-guided prompting improves prediction quality.
- **Fallback Correction Loop:** Detect failures using model confidence + action history. automatically re-prompt the model to “rethink” the step.
- **Human-in-the-Loop (HITL):** Use confidence thresholds to trigger human confirmation for low-confidence predictions.



Base Paper Implementation

- **Multimodal Min2web Loader:** Dataset loader used to load single task or batch of task from the different .parquet files .
- **Input Preparation:** Action Generation required specific input which is provided using input preparation layer.
- **Action Generation:** Core reasoning module. It generate textual step for defined user instruction using MLLM.
- **Action Grounding:** Map predicted plan to actual element using different methods.
- **Evaluation:** Evaluation used to compare the Outcome and ground truth for further analysis.



Base Paper Results

```

=====
2026-02-17 13:16:54,144 INFO: SeeAct Evaluation Summary
2026-02-17 13:16:54,144 INFO: =====
2026-02-17 13:16:54,144 INFO:
  Tasks evaluated : 31
2026-02-17 13:16:54,144 INFO: Total steps : 107
2026-02-17 13:16:54,144 INFO:
  — Step-Level Metrics —
2026-02-17 13:16:54,145 INFO: Element Accuracy : 5.61% (6/107)
2026-02-17 13:16:54,145 INFO: Operation F1 : 59.81% (64/107)
2026-02-17 13:16:54,145 INFO: Value Accuracy : 89.72% (96/107)
2026-02-17 13:16:54,145 INFO: Step Success Rate: 3.74% (4/107)
2026-02-17 13:16:54,145 INFO:
  — Task-Level Metrics —
2026-02-17 13:16:54,145 INFO: Offline0 (strict) : 9.68% (3/31)
2026-02-17 13:16:54,145 INFO: Offline1 (tolerance): 22.58% (7/31)
2026-02-17 13:16:54,146 INFO:
  Per Task Breakdown
  
```

Note*: results are generated with low detail mode

Table 2: Performance of different models. All models under SEEACT utilize “Choices” for grounding. Methods with * mark are conducted on a subset with 30 tasks for each task split.

Model	Cross-Task			Cross-Website			Cross-Domain		
	Ele. Acc	Op. F1	Step SR	Ele. Acc	Op. F1	Step SR	Ele. Acc	Op. F1	Step SR
Supervised Fine-Tuning									
FLAN-T5-XL	57.1	75.7	53.5	43.8	67.7	41.1	41.4	65.9	38.9
BLIP-2-T5-XL	50.1	77.0	47.0	39.4	69.3	37.0	41.2	69.3	38.9
In-Context Learning									
GPT-3.5*	19.4	59.2	16.8	14.9	56.5	14.1	25.2	57.9	24.1
GPT-4*	40.8	63.1	32.3	30.2	61.0	27.0	35.4	61.9	29.7
COGAGENT	22.4	53.0	17.6	18.4	42.2	13.4	20.6	42.0	15.5
SEEACT									
– LLaVA-1.5	9.7	65.6	8.1	9.1	60.8	7.5	10.9	63.9	8.5
– GEMINI PRO VISION	21.5	67.7	19.6	17.1	61.3	15.4	20.7	64.3	18.0
– GPT-4V	46.4	73.4	40.2	38.0	67.8	32.4	42.4	69.3	36.8
– GPT-4V-Oracle*	66.4	79.2	61.9	69.5	78.9	65.0	72.8	73.6	62.1

Table 3: Step success rate (%) of GPT-4V on a subset of 30 tasks for each task split with different grounding methods. “Attributes”, “Choices”, “Annotation”, and “Oracle” refer to element grounding via Element Attributes, Textual Choices, Image Annotation, and Human Annotation, respectively, as described in subsection 2.3.

Grounding	Cross-Task	Cross-Website	Cross-Domain
SEEACT _{Attribute}	16.1	12.1	19.0
SEEACT _{Annotation}	20.3	13.9	23.7
SEEACT _{Choice}	39.1	32.7	42.0
SEEACT _{Oracle}	61.9	65.0	62.1

Note*: results are generated with high detail mode

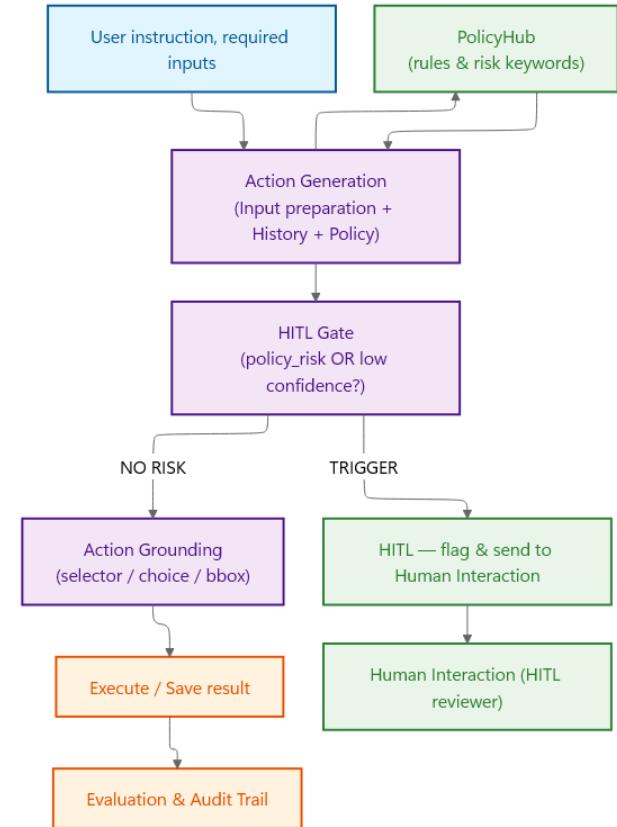
Source: [5]

Table 4: Whole task success rate (%) under both offline and online evaluation. Offline₀ and Offline₁ refer to no tolerance for error at any step and allowing for error at one step, respectively.

	Offline ₀	Offline ₁	Online
FLAN-T5-XL	4.4	24.4	8.9
GPT-4	1.1	12.2	13.3
SEEACT _{Choice}	3.3	12.2	37.8
SEEACT _{Oracle}	13.3	27.8	51.1

Improvement

- **PolicyHub:** Structured set of safety rules, contain default rules, user can add required rules in policy.
- **Risk-based HITL Gate:** Dedicated HITL Gate which filter risky or low confident actions.
- **Policy Risk:** Quantify of how risky an action might be according to predefined policies.
- **Confidence :** Model confidence to decide whether an action is correct or safe.
- **Adaptive Grounding:** HITL will block further grounding stage as generator has already planned what to do.



Improvement

- **SeeAct:**

$$at = \pi(st, T, \{a1, a2, \dots, at-1\})$$

$$st+1 = S(at) = \{ht+1, it+1\}$$

Where:

at = action at time t

s = site specific input (html + image)

π = Multimodel LLM

T = user defined task

- **Our's:**

$$At = \pi(st, T, \{a1, a2, \dots, at-1\}, pt)$$

$$st+1 = S(at) = \{ht+1, it+1\}$$

Where:

at = action at time t

s = site specific input (html + image)

π = Multimodel LLM

T = user defined task

pt = Policy rules

$At \in (at, c, r, hr)$

c = confidence $c \in [0,100]$,

$r \in \{0,1\}$,

hr = hitl_reason

Improvement - Algorithm

$A_t = \pi(st, T, \{a_1, a_2, \dots, a_{t-1}\}, pt)$

$at, c, r, hr = A_t$

matched = compare action plan and policy keywords

if ($r > 0$ or matched or $c < \text{confidence}$){

$r = 1$ #policy_risk detected

}

if ($r == 1$) {

 HITL value triggered, as wait for human interaction for step

} else {

 Executer further steps from grounding

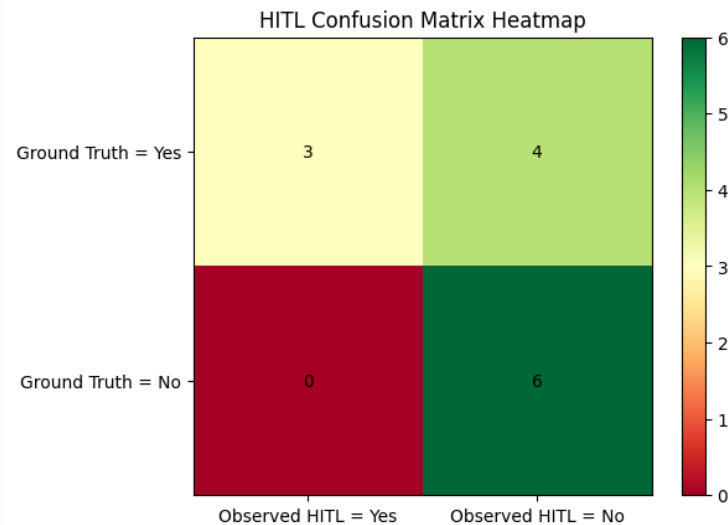
}

Implementation & Initial Result

*Initial results are computed on 13 live websites (7 required HITL, 6 not required HITL)

Metric	Value
HITL Recall	42.85%
HITL Precision	100%
HITL Accuracy	69.23%
False Positive Rate	0%
False Negative Rate	57.14%

- The system is able in triggering HITL.
- It avoids unnecessary human interruptions (0 false positives),
- but misses some high-risk cases (false negatives),
- which we aim to improve via expanded risk keywords and site-level overrides.



Future Work

- Gather and collect HITL results from different previous researches.
- Use their dataset evaluation strategy on our defined architecture and collect result data.
- Compute comparative results summary and tables.
- Draft research paper around that results.
- Search about publication and try for publication of our results.

Conclusion

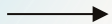
- In recent advancement we have seen that multimodal input (Screenshot + DOM) is essential for accurate UI action prediction.
- Preliminary tests show that lightweight models like Qwen2-VL-2B unable to understand and produce action plan for given user instruction.
- Initial result over novelty showcase high false negative rate which showcase failure over HITL demanding steps and need improvement.
- Implemented methodology use fixed confidence value which further can be update with adaptive learning.

References

- [1] C. Zhang, S. He, J. Qian, B. Li, L. Li, S. Qin, Y. Kang, M. Ma, G. Liu, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, "Large language model-brained gui agents: A survey," 2025.
- [2] F. Tang, H. Xu, H. Zhang, S. Chen, X. Wu, Y. Shen, W. Zhang, G. Hou, Z. Tan, Y. Yan, K. Song, J. Shao, W. Lu, J. Xiao, and Y. Zhuang, "A survey on (m)llm-based gui agents," 2025.
- [3] L. Ning, Z. Liang, Z. Jiang, H. Qu, Y. Ding, W. Fan, X. yong Wei, S. Lin, H. Liu, P. S. Yu, and Q. Li, "A survey of webagents: Towards next-generation ai agents for web automation with large foundation models," 2025.
- [4] S. Wang, W. Liu, J. Chen, Y. Zhou, W. Gan, X. Zeng, Y. Che, S. Yu, X. Hao, K. Shao, B. Wang, C. Wu, Y. Wang, R. Tang, and J. Hao, "Gui agents with foundation models: A comprehensive survey," 2025.
- [5] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "Gpt-4v(ision) is a generalist web agent, if grounded," 2024.
- [6] G. Verma, R. Kaur, N. Srishankar, Z. Zeng, T. Balch, and M. Veloso, "Adaptagent: Adapting multimodal web agents with few-shot learning from human demonstrations," 2024.

References

- [7] D. Zhang, B. Rama, J. Ni, S. He, F. Zhao, K. Chen, A. Chen, and J. Cao, “Litewebagent: The open-source suite for vlm-based web-agent applications,” 2025.
- [8] I. L. Long, X. Liu, Y. Chen, H. Lai, S. Yao, P. Shen, H. Yu, Y. Dong, and J. Tang, “Openwebagent: An open toolkit to enable web agents on large language models,” in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), pp. 72–81, 2024.
- [9] H. Luo, J. Kuang, W. Liu, Y. Shen, J. Luan, and Y. Deng, “Browsing like human: A multimodal web agent with experiential fast-and-slow thinking,” in Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 14232–14251, 2025.
- [10] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2web: Towards a generalist agent for the web,” 2023.



Do have any questions?

Thanks!